

Министерство науки и высшего образования Российской
Федерации

ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ АВТОНОМНОЕ
ОБРАЗОВАТЕЛЬНОЕ
УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ

НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ
УНИВЕРСИТЕТ
ИТМО
ITMO University

ЛАБОРАТОРНАЯ РАБОТА №1

По дисциплине Алгоритмы и структуры данных

Тема работы Знакомство с графами

Обучающийся Черненко Андрей Дмитриевич

Факультет Факультет прикладной информатики

Группа K4111c

Образовательная программа Разработка и эксплуатация
промышленного бэкенда

Обучающийся	_____	_____	<u>Черненко А. Д.</u>
	(дата)	(подпись)	(Ф.И.О.)

Руководитель	_____	_____	<u>Мусаев А. А.</u>
	(дата)	(подпись)	(Ф.И.О.)

Содержание

ВВЕДЕНИЕ	2
1 ОСНОВНОЕ ЗАДАНИЕ	2
1.1 Задание 1. Бинарный поиск	2
1.1.1 Условие	2
1.1.2 Решение	2
1.1.3 Принцип работы	3
1.2 Задание 2. Угадывание студента	3
1.2.1 Условие	3
1.2.2 Решение	4
1.2.3 Принцип работы	4
1.3 Задание 3. Граф дерева решений	5
1.3.1 Условие	5
1.3.2 Решение	5
1.3.3 Тип графа	6
2 LEETCODE	6
2.1 Задача 200. Number of Islands	6
2.1.1 Условие	6
2.1.2 Решение	7
2.1.3 Принцип работы	8
2.2 Задача 207. Course Schedule	8
2.2.1 Условие	8
2.2.2 Решение	9
2.2.3 Принцип работы	10
ЗАКЛЮЧЕНИЕ	11

ВВЕДЕНИЕ

Целью данной лабораторной работы является знакомство с графами и их применением для решения прикладных задач. В ходе работы рассматриваются базовые алгоритмы поиска в упорядоченных структурах, структура данных «словарь» и построение дерева решений как частного случая ориентированного графа.

В работе решаются следующие задачи:

1. реализация бинарного поиска с подсчётом количества шагов;
2. построение словаря характеристик студентов учебной группы и реализация программы-угадайки;
3. построение графа для задачи угадывания и определение его типа;
4. решение двух задач среднего уровня по теме «Графы» на платформе LeetCode.

Все программы реализованы на языке C# с использованием .NET 8.

1 ОСНОВНОЕ ЗАДАНИЕ

1.1 Задание 1. Бинарный поиск

1.1.1 Условие

Дан отсортированный по возрастанию массив целых чисел. Требуется реализовать алгоритм бинарного поиска, который по заданному числу определяет его позицию в массиве. Результатом работы программы должно быть количество шагов, потребовавшихся для нахождения числа.

1.1.2 Решение

```
1 public static (int index, int steps) BinarySearch(int[] array, int target)
2 {
3     int left = 0;
4     int right = array.Length - 1;
5     int steps = 0;
```

```

6
7  while (left <= right)
8  {
9      steps++;
10     int mid = left + (right - left) / 2;
11
12     if (array[mid] == target)
13         return (mid, steps);
14
15     if (array[mid] < target)
16         left = mid + 1;
17     else
18         right = mid - 1;
19 }
20
21 return (-1, steps);
22 }

```

Листинг 1 – Решение задачи 1

1.1.3 Принцип работы

1. Задаём границы поиска: `left = 0`, `right = length - 1`.
2. На каждом шаге вычисляем середину `mid` как `left + (right - left) / 2`, чтобы избежать переполнения.
3. Сравниваем значение в середине с искомым:
 - (a) если равны — элемент найден, возвращаем индекс и число шагов;
 - (b) если середина меньше искомого — сдвигаем `left` вправо;
 - (c) если больше — сдвигаем `right` влево.
4. Если `left > right` — элемент отсутствует в массиве.

Сложность алгоритма по времени — $O(\log n)$, по памяти — $O(1)$.

1.2 Задание 2. Угадывание студента

1.2.1 Условие

Для учебной группы составить словарь, описывающий характеристики каждого студента. Реализовать программу, которая по ответам на вопросы угадывает загаданного студента.

1.2.2 Решение

Модель студента и вопрос описываются следующими типами:

```
1 public record Student(  
2     string FullName,  
3     bool IsSmoking,  
4     bool IsBrunette,  
5     bool IsItmoBachelor,  
6     bool IsSport,  
7     bool IsPets,  
8     string InterestingFact);  
9  
10 public sealed record Question(string Text, Func<Student, bool> Predicate);
```

Листинг 2 – Модель данных

Алгоритм угадывания выбирает на каждом шаге вопрос, который делит текущее множество кандидатов наиболее равномерно:

```
1 public Question? ChooseNextQuestion()  
2 {  
3     Question? best = null;  
4     var bestScore = int.MaxValue;  
5  
6     foreach (var q in questions)  
7     {  
8         if (_askedTexts.Contains(q.Text)) continue;  
9  
10        var yes = _candidates.Count(q.Predicate);  
11        var no = _candidates.Count - yes;  
12        var score = Math.Max(yes, no);  
13  
14        if (score >= bestScore) continue;  
15  
16        bestScore = score;  
17        best = q;  
18    }  
19  
20    return best;  
21 }
```

Листинг 3 – Выбор лучшего вопроса

1.2.3 Принцип работы

1. На старте множество кандидатов — все студенты группы.

2. Перед каждым вопросом для каждого ещё не заданного вопроса вычисляется оценка $\text{score} = \max(\text{yes}, \text{no})$, где **yes** и **no** — количество кандидатов, ответивших «Да» и «Нет».
3. Выбирается вопрос с минимальной оценкой, то есть тот, который в худшем случае оставит меньше всего кандидатов.
4. После ответа пользователя множество кандидатов фильтруется, вопрос помечается как заданный.
5. Цикл продолжается, пока не останется один кандидат или не закончатся вопросы.

Временная сложность алгоритма — $O(Q^2 \cdot N)$, где N — число студентов, Q — число вопросов. По памяти — $O(N + Q)$.

1.3 Задание 3. Граф дерева решений

1.3.1 Условие

Составить граф для задания №2. Определить, к какому типу относится данный граф.

1.3.2 Решение

Процесс угадывания студента представляется в виде дерева решений. Внутренние узлы — вопросы с двумя исходящими рёбрами («Да» и «Нет»), листья — конкретные студенты. Граф приведён на рисунке 1.

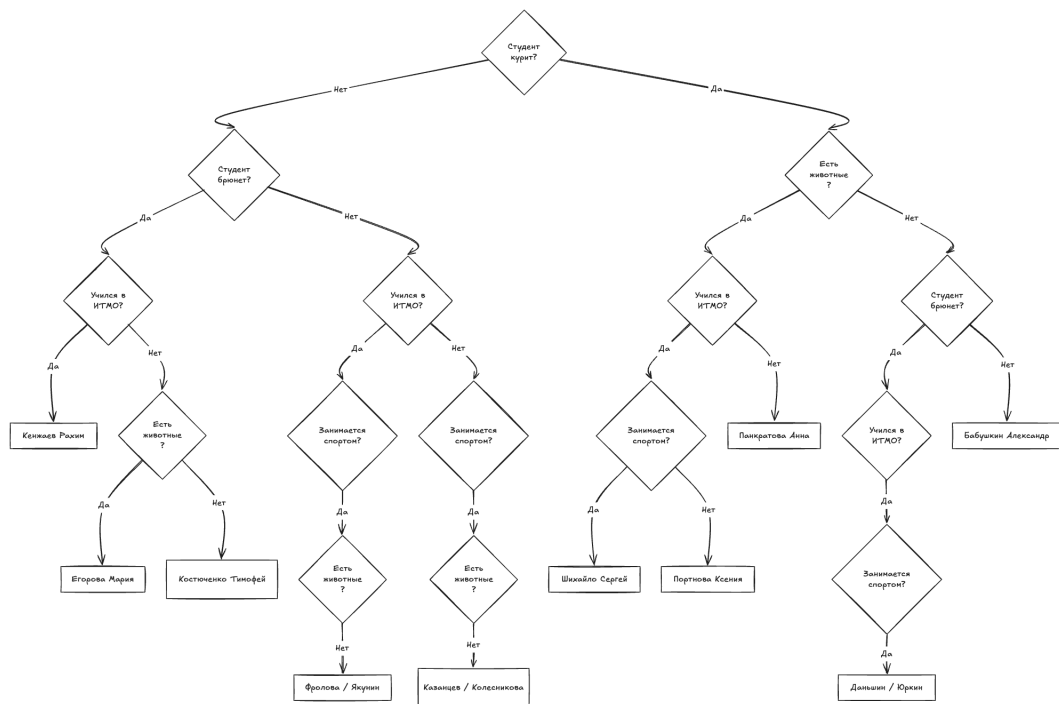


Рисунок 1 – Дерево решений для игры «Угадай студента»

1.3.3 Тип графа

Построенный граф является:

- **ориентированным** — рёбра имеют направление от вопроса к следующему вопросу или к студенту;
- **бинарным** — у каждого внутреннего узла ровно два исходящих ребра;
- **ациклическим и связным** — циклов нет, все вершины достижимы из корня.

2 LEETCODE

2.1 Задача 200. Number of Islands

2.1.1 Условие

Дана двумерная сетка **grid** размера $M \times N$, состоящая из символов '1' (суша) и '0' (вода). Островом называется группа смежных по горизон-

тали или вертикали клеток суши, окружённая водой. Требуется определить количество островов в сетке.

2.1.2 Решение

```
1 public class Solution
2 {
3     public int NumIslands(char [][] grid)
4     {
5         var islandsCount = 0;
6
7         for (int i = 0; i < grid.Length; i++)
8         {
9             for (int j = 0; j < grid[0].Length; j++)
10            {
11                if (grid[i][j] != '1')
12                    continue;
13
14                islandsCount++;
15                DFS(grid, i, j);
16            }
17        }
18
19        return islandsCount;
20    }
21
22    public void DFS(char [][] grid, int i, int j)
23    {
24        if (i < 0 || i >= grid.Length || j < 0 || j >= grid[0].Length)
25            return;
26
27        if (grid[i][j] != '1')
28            return;
29
30        grid[i][j] = '0';
31
32        DFS(grid, i + 1, j);
33        DFS(grid, i - 1, j);
34        DFS(grid, i, j + 1);
35        DFS(grid, i, j - 1);
36    }
37 }
```

Листинг 4 – Решение задачи 200

2.1.3 Принцип работы

Задача сводится к подсчёту числа связных компонент в графе, где вершинами являются клетки суши, а рёбрами — отношения смежности между ними.

1. Внешние циклы перебирают все клетки сетки.
2. Если встречается клетка '1', это означает начало нового острова. Счётчик увеличивается на единицу, после чего запускается обход в глубину.
3. Функция DFS рекурсивно обходит все смежные клетки суши и «затпливает» их, заменяя символ '1' на '0'. Это необходимо, чтобы внешний цикл не посчитал клетки одного и того же острова повторно.
4. Базовый случай рекурсии — выход за границы сетки или попадание в клетку, которая уже не является сушей.

Ключевая идея метода: после обработки острова все его клетки превращаются в воду, поэтому при дальнейшем переборе они игнорируются. Каждый остров получает ровно одно инкрементирование счётчика — в момент, когда алгоритм впервые встречает одну из его клеток.

Сложность алгоритма по времени — $O(M \cdot N)$. По памяти — $O(M \cdot N)$ в худшем случае.

2.2 Задача 207. Course Schedule

2.2.1 Условие

Дано общее количество курсов `numCourses` и список предварительных требований `prerequisites`, где `prerequisites[i] = [a, b]` означает, что для прохождения курса *a* необходимо предварительно пройти курс *b*. Требуется определить, возможно ли завершить все курсы. Фактически задача сводится к проверке наличия цикла в ориентированном графе зависимостей.

2.2.2 Решение

```
1 public class Solution
2 {
3     private List<List<int>> _adj;
4     private int[] _state;
5
6     public bool CanFinish(int numCourses, int[][] prerequisites)
7     {
8         _adj = new List<List<int>>();
9         for (int i = 0; i < numCourses; i++)
10             _adj.Add(new List<int>());
11
12         foreach (var pair in prerequisites)
13         {
14             int course = pair[0];
15             int prerequisite = pair[1];
16             _adj[course].Add(prerequisite);
17         }
18
19         _state = new int[numCourses];
20
21         for (int i = 0; i < numCourses; i++)
22         {
23             if (_state[i] == 0 && HasCycle(i))
24                 return false;
25         }
26
27         return true;
28     }
29
30     private bool HasCycle(int v)
31     {
32         if (_state[v] == 1) return true;
33         if (_state[v] == 2) return false;
34
35         _state[v] = 1;
36
37         foreach (int neighbor in _adj[v])
38         {
39             if (HasCycle(neighbor)) return true;
40         }
41
42         _state[v] = 2;
43         return false;
44     }
45 }
```

2.2.3 Принцип работы

Задача сводится к обнаружению цикла в ориентированном графе, где вершинами являются курсы, а рёбра отражают зависимости между ними. Все курсы можно пройти тогда и только тогда, когда в графе нет цикла: при наличии цикла ни один из входящих в него курсов нельзя пройти первым.

Для поиска цикла используется обход в глубину с тремя состояниями вершин:

- (значение 0) — вершина ещё не посещалась;
- (значение 1) — вершина находится в стеке рекурсии, обработка ещё не завершена;
- (значение 2) — вершина и все её потомки полностью обработаны, цикла через неё нет.

Алгоритм работает следующим образом:

1. Строится список смежности: для каждого курса сохраняется список его предварительных требований.
2. Внешний цикл перебирает все курсы, поскольку граф может быть несвязным. Для каждой вершины со значением 0 запускается рекурсивная проверка.
3. Функция `HasCycle` сначала проверяет состояние вершины: если она имеет значение 1 — цикл найден, если значение 2 — обработка уже завершена и результат отрицательный.
4. Текущая вершина получает 1, после чего рекурсивно обходятся все её соседи.
5. Если ни от одного из соседей не пришло положительного ответа, вершине присваивается значение 2, и возвращается `false`.

Сложность алгоритма по времени — $O(V + E)$, где V — количество курсов, E — количество зависимостей. По памяти — $O(V + E)$.

ЗАКЛЮЧЕНИЕ

В ходе выполнения лабораторной работы были изучены основы работы с графами и деревьями решений. Реализован алгоритм бинарного поиска с подсчётом шагов, построена программа-угадайка на основе словаря характеристик студентов, а также составлен граф процесса угадывания и определён его тип.

Дополнительно на платформе LeetCode решены две задачи среднего уровня по теме «Графы»: подсчёт числа островов на сетке через обход в глубину и проверка возможности завершить все курсы через поиск цикла в ориентированном графе.